

Reporte Técnico de Actividades Práctico-Experimentales Nro. 001

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Vasquez C. Jostin X.
Asignatura	Sistemas Digitales
Ciclo	5 A
Unidad	1
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	001
Tipo	Grupal
Título de la Práctica	Uso de Wokwi y Microchip Studio en microcontroladores AVR: arquitectura, puertos de E/S y operaciones a nivel de bit
Nombre del Docente	Luis Darío Sinche C
Fecha	Viernes, 2 de Octubre del 2026
Horario	11h30 – 13h30
Lugar	Laboratorio
Tiempo planificado en el Sílabo	2 horas

2. Objetivo(s) de la Práctica

- Objetivo general. Reconocer la arquitectura del microcontrolador ATmega328P y controlar sus periféricos de entradas/salidas digitales escribiendo y leyendo directamente sus registros mediante operaciones a nivel de bit, usando simulación en Wokwi y Microchip Studio. Objetivos específicos: Relacionar los pines de la placa Arduino UNO con los puertos y bits del ATmega328P (PB, PC, PD). Configurar entradas y salidas digitales con los registros DDRx, PORTx y PINx usando máscaras (AND, OR, XOR, NOT y desplazamientos). Comparar el acceso mediante la API de Arduino con el acceso directo a registros, en tiempo de ejecución. Identificar en el código ensamblador generado las instrucciones lógicas y de manejo de bits del set de instrucciones AVR (SBI, CBI, SBIS, IN, OUT, ORI, EOR).

3. Materiales, Reactivos

- No se requieren componentes físicos: toda la práctica es simulada. Componentes virtuales usados en Wokwi: 1 Arduino UNO (ATmega328P a 16 MHz). 4 LEDs (rojo, amarillo, verde y azul) y 4 resistencias de 220 Ω . 1 pulsador.

4. Equipos y Herramientas

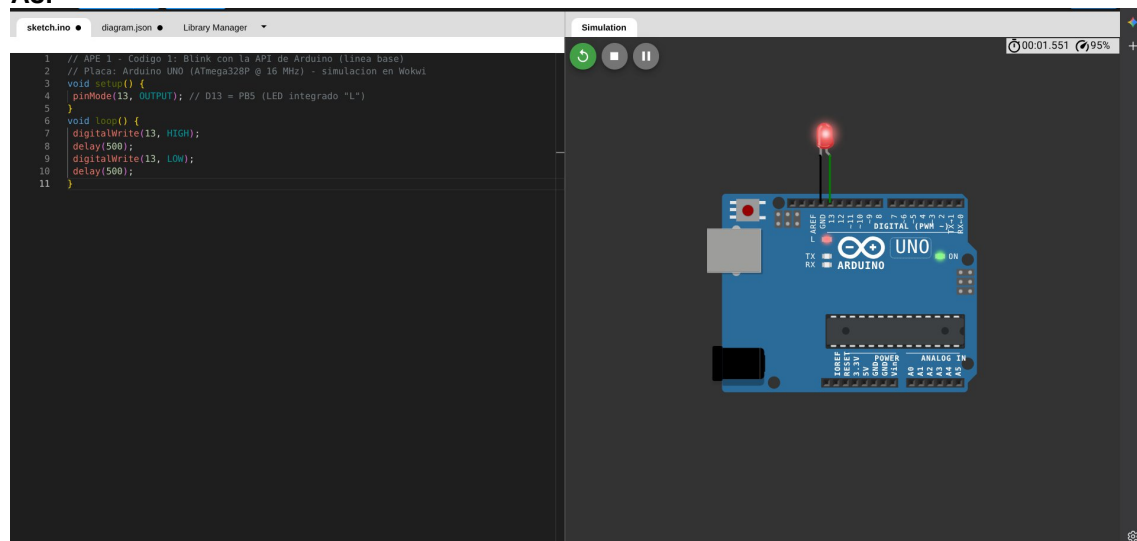
- Computador con navegador web y conexión a internet.
- Wokwi (<https://wokwi.com>), con cuenta gratuita para guardar y compartir los proyectos. FEIRNNR - Carrera de Computación Microchip Studio 7 (solo Windows), con el simulador del ATmega328P.
- Si no dispone de Windows, use Compiler Explorer (<https://godbolt.org>) para la Parte E.
- Hoja de datos del ATmega328P y AVR Instruction Set Manual (Microchip Technology). Cuenta de GitHub y acceso al EVA para la entrega.

5. Procedimiento / Metodología

1. Preparé mi entorno de trabajo virtual usando las plataformas Wokwi y Microchip Studio, ya que toda la práctica fue simulada sin usar componentes físicos reales.
2. Primero, conecté LEDs y un pulsador virtuales a la placa Arduino UNO. Luego, programé el circuito para comparar la forma normal de Arduino con la modificación directa de los "registros" internos del microcontrolador. Al medir la velocidad, comprobé que programar usando los registros es muchísimo más rápido (0.220 microsegundos por escritura frente a 3.395). También analicé cómo mi código se convertía paso a paso en lenguaje básico de máquina.
3. Junté los datos obtenidos para completar las tablas de resultados. Con esto listo, respondí las preguntas de control y escribí mis conclusiones sobre las ventajas de simular los circuitos para no quemar piezas y la rapidez de programar directamente el hardware.

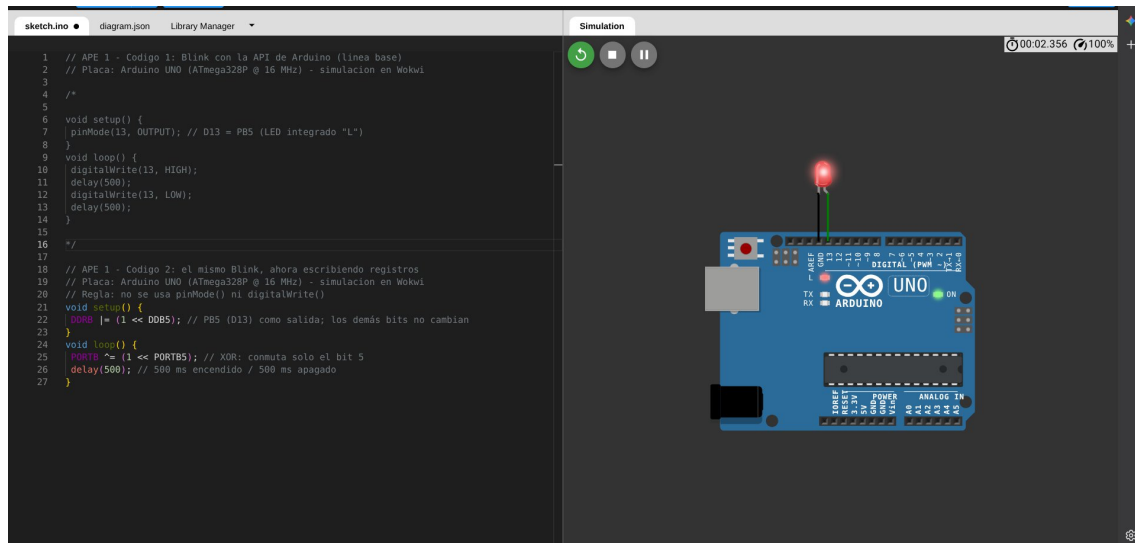
6. Resultados

A3:



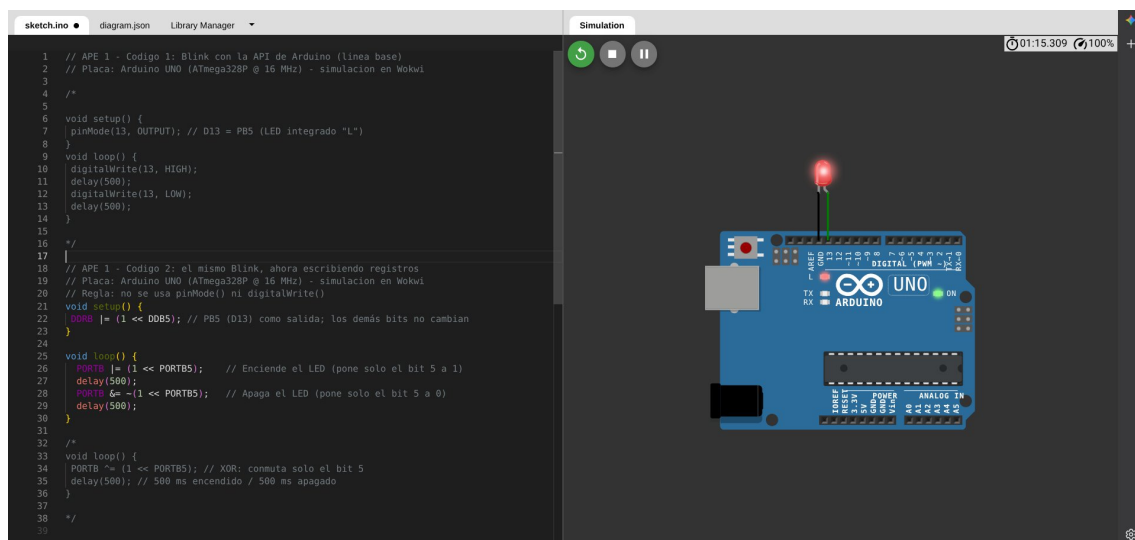
<https://wokwi.com/projects/477100885147379713>

B1:



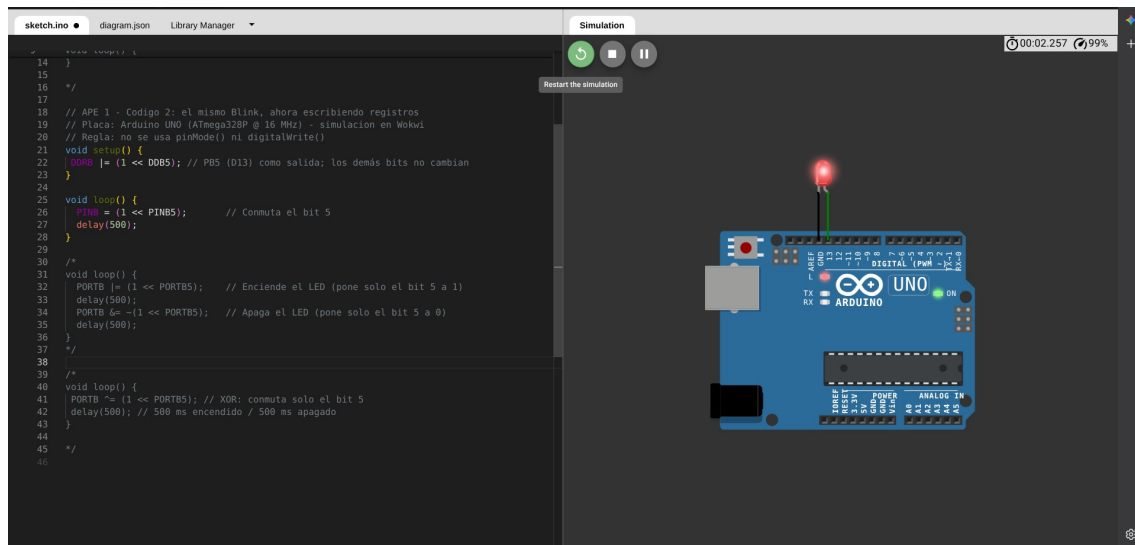
<https://wokwi.com/projects/477100885147379713>

B2:



<https://wokwi.com/projects/477100885147379713>

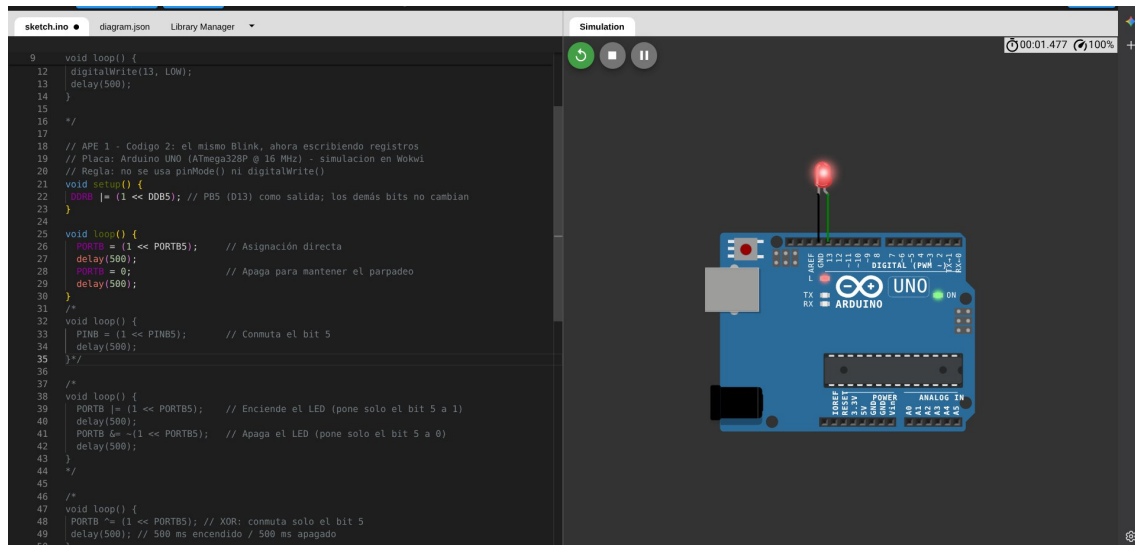
B3:



<https://wokwi.com/projects/477100885147379713>

La característica del ATmega328p que lo permite es que escribir un 1 en un bit de PINx conmuta el bit correspondiente de PORTx.

B4:



<https://wokwi.com/projects/477100885147379713>

C2:

sketch.ino

diagram.json

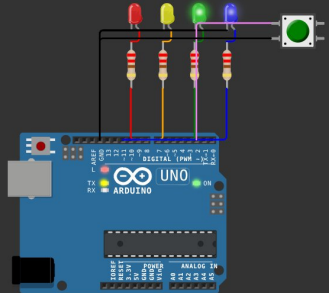
Library Manager

```

14 uint8_t bitAnterior = 1; // con pull-up, en reposo se lee 1
15 unsigned long tAnterior = 0;
16
17 void imprimirBin8(const char* etiqueta, uint8_t v) {
18   Serial.print(etiqueta);
19   for (int8_t i = 7; i >= 0; i--) { // extrae el bit i
20     Serial.print((v >> i) & 1);
21   }
22   Serial.println();
23 }
24
25 void escribirLeds(uint8_t valor) {
26   // leer-modificar-escribir: solo cambian PB0..PB3; PB4..PB7 se conservan
27   PORTB = (PORTB & ~MASK_LEDS) | (valor & MASK_LEDS);
28 }
29
30 void demoOperaciones() {
31   uint8_t a = 0b1001100;
32   uint8_t b = 0b10101010;
33   imprimirBin8("a = ", a);
34   imprimirBin8("b = ", b);
35   imprimirBin8("a & b = ", a & b);
36   imprimirBin8("a | b = ", a | b);
37   imprimirBin8("a ^ b = ", a ^ b);
38   imprimirBin8("~a = ", (~a));
39   imprimirBin8("a << 1 = ", (uint8_t)(a << 1));
40   imprimirBin8("a >> 2 = ", (uint8_t)(a >> 2));
41   Serial.print("a sin cast, en BIN: ");
42   Serial.println(a, BIN); // ver pregunta de control 6
43 }
44
45 void setup() {
46   DDRB &= ~(1 << DD02); // PD2 entrada (bit DOR = 0)
47   PORTB |= (1 << PORTD2); // pull-up interna (bit PORT = 1)
48   DDRB |= MASK_LEDS | (1 << DD05); // salidas: PB0..PB3 y PB5
49   Serial.begin(9600);
50   Serial.println(F("APE 1 - Laboratorio de bits"));
51   demoOperaciones();
52   escribirLeds(patron);
53 }

```

Simulation



Modo: 0

PORTB = 00100001

Modo: 1

PORTB = 00000010

PORTB = 00100100

PORTB = 00001000

PORTB = 00100001

PORTB = 00000010

<https://wokwi.com/projects/477101421735351297>

WOKWI

SAVE

SHARE

APET_Vasquez_Calderon#2

Docs

SIGN UP

sketch.ino

diagram.json

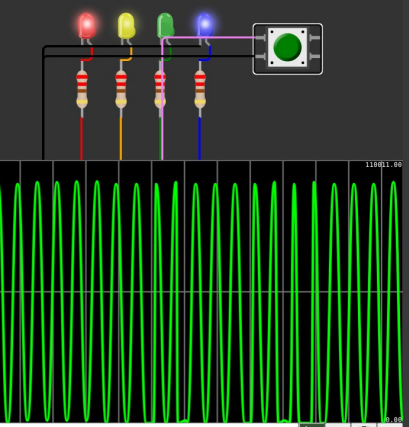
Library Manager

```

1 // APE 1 - Laboratorio de bits en los puertos B y D
2 // Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulacion en Wokwi
3 // Libs : ninguna (avr/io.h llega incluido por Arduino.h)
4 // Pines: PB0..PB3 (D0..D13) = LEDs; PD2 (D2) = pulsador a GND; PB5 (D13) = LED L
5 // Regla: NO se usa pinMode(), digitalWrite() ni digitalRead()
6
7 const uint8_t MASK_LEDS = 0x0F; // 0b00001111 -> PB0..PB3
8 const uint8_t NUM_MODOS = 3;
9 const uint8_t PATRON_INICIAL[NUM_MODOS] = {0b0000, 0b0001, 0b0101};
10 const unsigned long PERIODO_MS = 300;
11
12 uint8_t modo = 0;
13 uint8_t patron = PATRON_INICIAL[0];
14 uint8_t bitAnterior = 1; // con pull-up, en reposo se lee 1
15 unsigned long tAnterior = 0;
16
17 void imprimirBin8(const char* etiqueta, uint8_t v) {
18   Serial.print(etiqueta);
19   for (int8_t i = 7; i >= 0; i--) { // extrae el bit i
20     Serial.print((v >> i) & 1);
21   }
22   Serial.println();
23 }
24
25 void escribirLeds(uint8_t valor) {
26   // leer-modificar-escribir: solo cambian PB0..PB3; PB4..PB7 se conservan
27   PORTB = (PORTB & ~MASK_LEDS) | (valor & MASK_LEDS);
28 }
29
30 void demoOperaciones() {
31   uint8_t a = 0b1001100;
32   uint8_t b = 0b10101010;
33   imprimirBin8("a = ", a);
34   imprimirBin8("b = ", b);
35   imprimirBin8("a & b = ", a & b);
36   imprimirBin8("a | b = ", a | b);
37   imprimirBin8("a ^ b = ", a ^ b);
38   imprimirBin8("~a = ", (~a));
39   imprimirBin8("a << 1 = ", (uint8_t)(a << 1));
40   imprimirBin8("a >> 2 = ", (uint8_t)(a >> 2));
41 }

```

Simulation



C3

Tenemos

$a = 11001100$
 $b = 10101010$

① AND $a \& b$

$$\begin{array}{r} 11001100 \\ \& 10101010 \\ \hline 10001000 \end{array}$$

② OR $a | b$

$$\begin{array}{r} 11001100 \\ | 10101010 \\ \hline 11101110 \end{array}$$

③ XOR $a \wedge b$

$$\begin{array}{r} 11001100 \\ \wedge 10101010 \\ \hline 01100110 \end{array}$$

④ Not $\sim a$

$a = 11001100$
 $\sim a = 00110011 //$

⑤ Desplazamiento $a \ll 1$

$$\begin{array}{r} 11001100 \ll 1 \\ 11001100 \\ \hline 10011000 // \end{array}$$

⑥ Desplazamiento $a \gg 2$

$$\begin{array}{r} 11001100 \gg 2 \\ 11001100 \gg 1 = 01100110 \\ 01100110 \gg 1 = 00110011 // \end{array}$$

C4:

MODO 1:

WOKWI

sketch.ino diagram.json Library Manager

```

1 // APE 1 - Código 3: Laboratorio de bits en los puertos B y D
2 // Placa: Arduino UNO (ATmega2560 @ 16 MHz) - simulación en Wokwi
3 // Libs: ninguna (avr/io.h llega incluido por Arduino.h)
4 // Pines: PB0..PB3 (D8..D11) = LEDs | PD2 (D2) = pulsador a GND | PB5 (D13) = LED L
5 // Regla: NO se usa pinMode(), digitalWrite() ni digitalRead()
6
7 const uint8_t MASK_LEDS = 0xF; // 0b00001111 -> PB0..PB3
8 const uint8_t NUM_MODOS = 3;
9 const uint8_t PATRON_INICIAL[NUM_MODOS] = {0b0000, 0b0001, 0b0101};
10 const unsigned long PERIODO_MS = 300;
11
12 uint8_t modo = 0;
13 uint8_t patron = PATRON_INICIAL[0];
14 uint8_t btnAnterior = 1; // con pull-up, en reposo se lee 1
15 unsigned long tAnterior = 0;
16
17 void imprimirBin8(const char* etiqueta, uint8_t v) {
18   Serial.print(etiqueta);
19   for (int8_t i = 7; i >= 0; i--) { // extrae el bit i
20     Serial.print((v >> i) & 1);
21   }
22   Serial.println();
23 }
24
25 void escribirLeds(uint8_t valor) {
26   // leer-modificar-escribir: solo cambian PB0..PB3; PB4..PB7 se conservan
27   PORTB = (PORTB & ~MASK_LEDS) | (valor & MASK_LEDS);
28 }
29
30 void demoOperaciones() {
31   uint8_t a = 0b1001100;
32   uint8_t b = 0b10101010;
33   imprimirBin8("a = ", a);
34   imprimirBin8("b = ", b);
35   imprimirBin8("a & b = ", a & b);
36   imprimirBin8("a | b = ", a | b);
37   imprimirBin8("a ^ b = ", a ^ b);
38   imprimirBin8("a ~ = ", ~(uint8_t)a);
39   imprimirBin8("a << 1 = ", (uint8_t)(a << 1));
40   imprimirBin8("a >> 2 = ", a >> 2);

```

Simulation

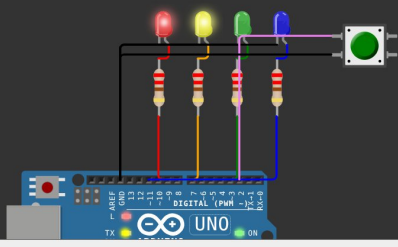
00:23.230 100%

Modo: 1

```

PORTB = 0b100010
PORTB = 0b0000100
PORTB = 0b101000
PORTB = 0b0000001
PORTB = 0b100010
PORTB = 0b0000100
PORTB = 0b101000
PORTB = 0b0000001
PORTB = 0b100010
PORTB = 0b0000100
PORTB = 0b101000

```



MODO 2:



1859



Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

WOKWI

sketch.ino diagram.json Library Manager

```
1 // APE 1 - Código 3: Laboratorio de bits en los puertos B y D
2 // Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulación en Wokwi
3 // Libs : ninguna (avr/io.h llega incluido por Arduino.h)
4 // Pines : PB0..PB3 (D8..D11) = LEDs | PD2 (D2) = pulsador a GND | PB5 (D13) = LED L
5 // Regla: NO se usa pinMode(), digitalWrite() ni digitalRead()
6
7 const uint8_t MASK_LEDS = 0x0F; // 0b00001111 -> PB0..PB3
8 const uint8_t NUM_MODOS = 3;
9 const uint8_t PATRON_INICIAL[NUM_MODOS] = {0b0000, 0b0001, 0b0101};
10 const unsigned long PERIODO_MS = 300;
11
12 uint8_t modo = 0;
13 uint8_t patron = PATRON_INICIAL[0];
14 uint8_t btnAnterior = 1; // con pull-up, en reposo se lee 1
15 unsigned long tAnterior = 0;
16
17 void imprimirBin8(const char* etiqueta, uint8_t v) {
18   Serial.print(etiqueta);
19   for (int8_t i = 7; i >= 0; i--) { // extrae el bit i
20     Serial.print((v >> i) & 1);
21   }
22   Serial.println();
23 }
24
25 void escribirLeds(uint8_t valor) {
26   // leer-modificar-escribir: solo cambian PB0..PB3; PB4..PB7 se conservan
27   PORTB = (PORTB & ~MASK_LEDS) | (valor & MASK_LEDS);
28 }
29
30 void demoOperaciones() {
31   uint8_t a = 0b11001100;
32   uint8_t b = 0b10101010;
33   imprimirBin8("a", a);
34   imprimirBin8("b", b);
35   imprimirBin8("a & b =", a & b);
36   imprimirBin8("a | b =", a | b);
37   imprimirBin8("a ^ b =", a ^ b);
38   imprimirBin8("a ~ =", (~a));
39   imprimirBin8("a << 1 =", (a << 1));
40   imprimirBin8("a >> 2 =", (a >> 2));
41 }
```

Simulation

00:44.604 100%

PORTB = 00100010
PORTB = 00000100
Modo: 2
PORTB = 00101010
PORTB = 00000101
PORTB = 00101010
PORTB = 00000101
PORTB = 00101010
PORTB = 00000101
PORTB = 00101010
PORTB = 00000101

MODO 3:

WOKWI

sketch.ino diagram.json Library Manager

```
1 // APE 1 - Código 3: Laboratorio de bits en los puertos B y D
2 // Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulación en Wokwi
3 // Libs : ninguna (avr/io.h llega incluido por Arduino.h)
4 // Pines : PB0..PB3 (D8..D11) = LEDs | PD2 (D2) = pulsador a GND | PB5 (D13) = LED L
5 // Regla: NO se usa pinMode(), digitalWrite() ni digitalRead()
6
7 const uint8_t MASK_LEDS = 0x0F; // 0b00001111 -> PB0..PB3
8 const uint8_t NUM_MODOS = 3;
9 const uint8_t PATRON_INICIAL[NUM_MODOS] = {0b0000, 0b0001, 0b0101};
10 const unsigned long PERIODO_MS = 300;
11
12 uint8_t modo = 0;
13 uint8_t patron = PATRON_INICIAL[0];
14 uint8_t btnAnterior = 1; // con pull-up, en reposo se lee 1
15 unsigned long tAnterior = 0;
16
17 void imprimirBin8(const char* etiqueta, uint8_t v) {
18   Serial.print(etiqueta);
19   for (int8_t i = 7; i >= 0; i--) { // extrae el bit i
20     Serial.print((v >> i) & 1);
21   }
22   Serial.println();
23 }
24
25 void escribirLeds(uint8_t valor) {
26   // leer-modificar-escribir: solo cambian PB0..PB3; PB4..PB7 se conservan
27   PORTB = (PORTB & ~MASK_LEDS) | (valor & MASK_LEDS);
28 }
29
30 void demoOperaciones() {
31   uint8_t a = 0b11001100;
32   uint8_t b = 0b10101010;
33   imprimirBin8("a", a);
34   imprimirBin8("b", b);
35   imprimirBin8("a & b =", a & b);
36   imprimirBin8("a | b =", a | b);
37   imprimirBin8("a ^ b =", a ^ b);
38   imprimirBin8("a ~ =", (~a));
39   imprimirBin8("a << 1 =", (a << 1));
40   imprimirBin8("a >> 2 =", (a >> 2));
41 }
```

Simulation

01:00.877 100%

PORTB = 00000101
PORTB = 00101010
Modo: 0
PORTB = 00000001
PORTB = 00100010
PORTB = 00000011
PORTB = 00100100
PORTB = 00000101
PORTB = 00100110
PORTB = 00000111
PORTB = 00101000

C6:

WOKWI

sketch.ino diagram.json Library Manager

```
1 // APE 1 - Código 3: Laboratorio de bits en los puertos B y D
2 // Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulación en Wokwi
3 // Libs : ninguna (avr/io.h llega incluido por Arduino.h)
4 // Pines : PB0..PB3 (D8..D11) = LEDs | PD2 (D2) = pulsador a GND | PB5 (D13) = LED L
5 // Regla: NO se usa pinMode(), digitalWrite() ni digitalRead()
6
7 const uint8_t MASK_LEDS = 0x0F; // 0b00001111 -> PB0..PB3
8 const uint8_t NUM_MODOS = 3;
9 const uint8_t PATRON_INICIAL[NUM_MODOS] = {0b0000, 0b0001, 0b0101};
10 const unsigned long PERIODO_MS = 300;
11
12 uint8_t modo = 0;
13 uint8_t patron = PATRON_INICIAL[0];
14 uint8_t btnAnterior = 1; // con pull-up, en reposo se lee 1
15 unsigned long tAnterior = 0;
16 void imprimirBin8(const char* etiqueta, uint8_t v) {
17   Serial.print(etiqueta);
18   for (int8_t i = 7; i >= 0; i--) {
19     Serial.print((v >> i) & 1);
20   }
21   Serial.println();
22 }
23 void escribirLeds(uint8_t valor) {
24   PORTB = valor;
25 }
26 void demoOperaciones() {
27   uint8_t a = 0b11001100;
28   uint8_t b = 0b10101010;
29   imprimirBin8("a =", a);
30   imprimirBin8("b =", b);
31   imprimirBin8("a & b =", a & b);
32   imprimirBin8("a | b =", a | b);
33   imprimirBin8("a ^ b =", a ^ b);
34   imprimirBin8("a ~ =", (~a));
35 }
```

Simulation

00:03.488 99%

PORTB = 00100101
PORTB = 00100110
PORTB = 00100111
PORTB = 00101000
PORTB = 00101001
PORTB = 00101010
PORTB = 00101011

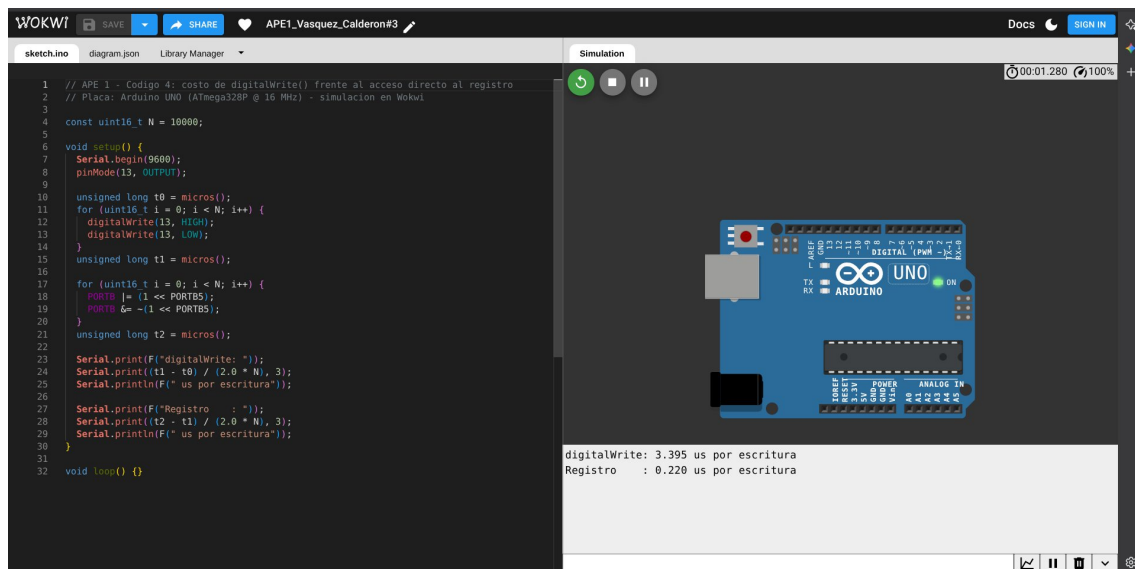
Al sustituir la función escribirLeds() por `PORTB = valor;`, el LED “L” de D13 (PB5) deja de parpadear de manera adecuada. Esto sucede porque la asignación directa altera todos los bits del registro `PORTB`, incluyendo PB5, aunque el valor solo tiene el patrón dirigido a PB0–PB3. Por otro lado, la versión original emplea una operación de lectura-modificación-escritura que posibilita modificar solo PB0–PB3 y mantener el estado de los otros bits. Esto está vinculado con B4, donde se notó que `PORTB = (1 << PORTB5)` puede alterar las demás salidas del puerto.

C7:

PIN	FUNCION	Bit DDRx	Bit PORTx	Bit PINx leído
D8 (PB0)	LED 0 encendido	1	1	—
D11 (PB3)	LED3 apagado	1	0	—
D2 (PD2)	Pulsador suelto	0	1	1
D2 (PD2)	Pulsador presionado	0	1	0
D13 (PB5)	LED “L” encendido	1	1	—

D1:

1ra ejecucion:



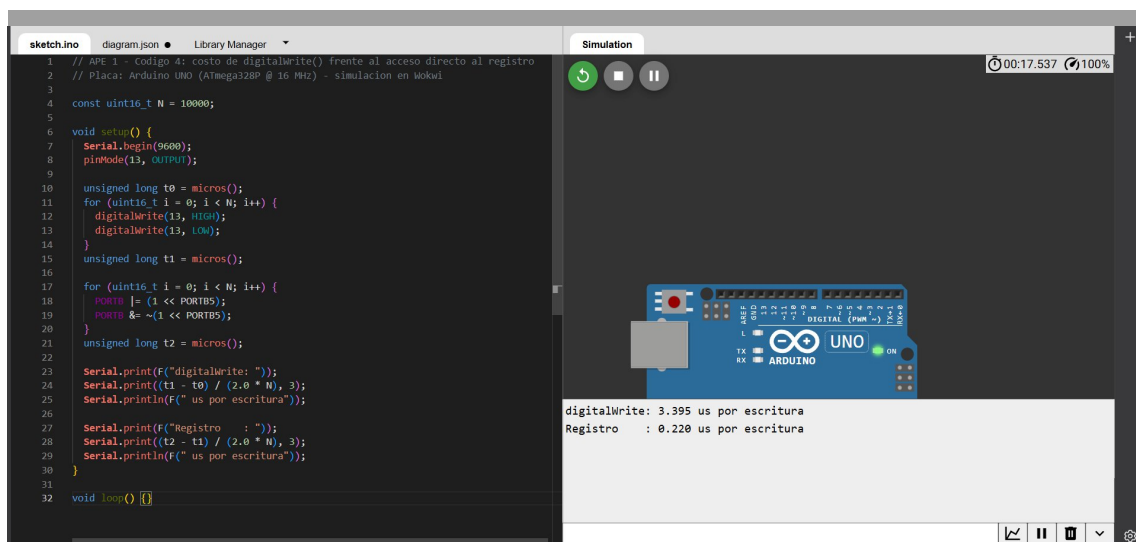
```

1 // APE 1 -Codigo 4: costo de digitalWrite() frente al acceso directo al registro
2 // Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulacion en Wokwi
3
4 const uint16_t N = 10000;
5
6 void setup() {
7   Serial.begin(9600);
8   pinMode(13, OUTPUT);
9
10  unsigned long t0 = micros();
11  for (uint16_t i = 0; i < N; i++) {
12    digitalWrite(13, HIGH);
13    digitalWrite(13, LOW);
14  }
15  unsigned long t1 = micros();
16
17  for (uint16_t i = 0; i < N; i++) {
18    PORTB |= (1 << PORTB5);
19    PORTB &= ~(1 << PORTB5);
20  }
21  unsigned long t2 = micros();
22
23  Serial.print(F("digitalWrite: "));
24  Serial.print((t1 - t0) / (2.0 * N), 3);
25  Serial.println(F(" us por escritura"));
26
27  Serial.print(F("Registro : "));
28  Serial.print((t2 - t1) / (2.0 * N), 3);
29  Serial.println(F(" us por escritura"));
30
31 }
32 void loop() {}

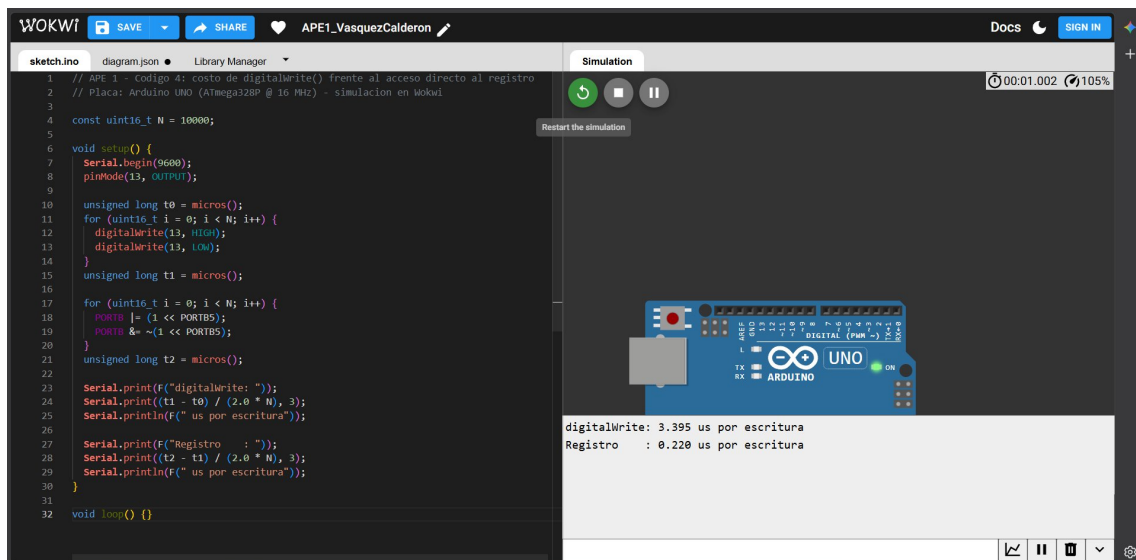
```

digitalWrite: 3.395 us por escritura
Registro : 0.220 us por escritura

2da ejecucion



3ra ejecucion



<https://wokwi.com/projects/477101768029796353>

Promedio final **3.395**

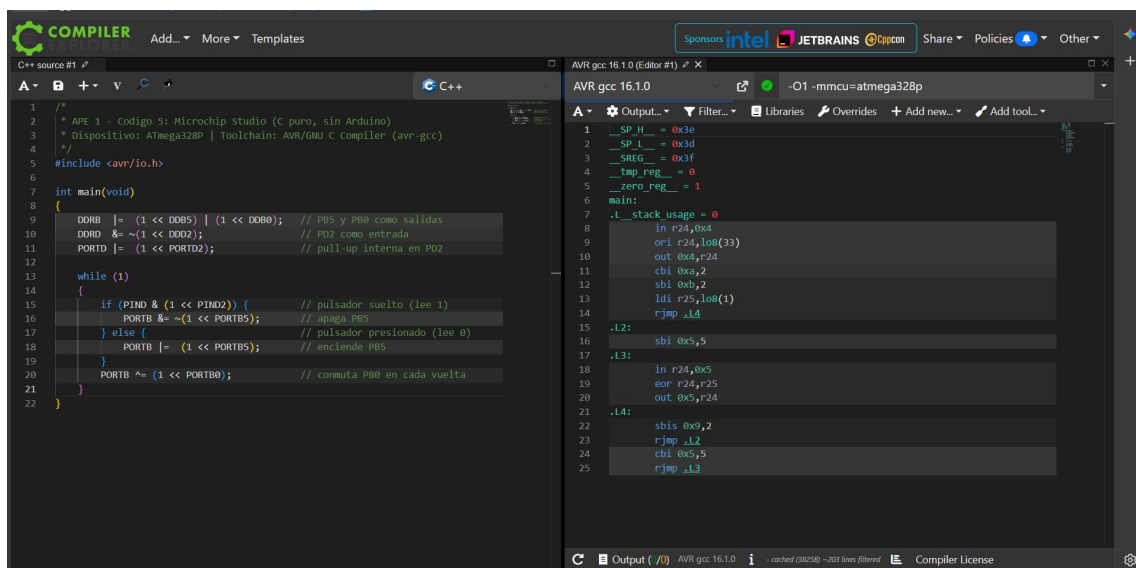


Tabla 4. Configuración de registros (para completar)

Pin	Función	Bit DDRx	Bit PORTx	Bit PINx leído
D8 (PB0)	LED0 encendido	1	0	—
D11 (PB3)	LED3 apagado	1	0	—
D2 (PD2)	Pulsador suelto	0	1	1
D2 (PD2)	Pulsador presionado	0	1	0
D13 (PB5)	LED "L" encendido	1	1	—

D2:

Tabla 5. Correspondencia C – ensamblador (para completar)

Línea en C	Instrucción(es) AVR	Ciclos
DDRB = (1 << DDB5) (1 << DDB0);	in r24,0x4 ori r24,lo8(33) out 0x4,r24	3 (1 por instrucción)
DDRD &= ~(1 << DDD2);	cbi 0xa,2	2
PORTD = (1 << PORTD2);	sbi 0xb,2	2
if (PIND & (1 << PIND2))	sbi 0x9,2	1 o 2
PORTB &= ~(1 << PORTB5);	cbi 0x5,5	2
PORTB = (1 << PORTB5);	sbi 0x5,5	2
PORTB ^= (1 << PORTB0);	in r24,0x eor r24,r25 out 0x5,r24	3 (1 por instrucción)

E2:



1859



Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

```
main.c
// Created: 4/18/2023 15:10:46
// Author: Higuely

#include <avr/io.h>

/*
 * APF 1 - Código 5: Microchip Studio (C gnu, sin Arduino)
 * dispositivo: ATmega328P | toolchain: AVR/GNU C Compiler [avr-gcc]
 */
#include <avr/io.h>

int main(void)
{
    DDRC |= (1 << DDB5) | (1 << DDB0); // PPS y PDB como salidas
    DDMD &= ~(1 << DDB2); // PDB como entrada
    PORTD |= (1 << PORTD2); // pull-up Jdscce en P02

    while (1)
    {
        if (PIND & (1 << PIND2)) { // pulsador suelto (lee 1)
            PORTB &= ~(1 << PORTB5); // apaga PPS
        } else { // pulsador presionado (lee 0)
            PORTB |= (1 << PORTB5); // enciende PPS
        }
        PORTB ^= (1 << PORTB0); // conmute PDB en cada vuelta
    }
}
```

Output

Show output from: Build

Using "RunOutputFileVerifyTask" task from assembly "C:\Program Files (x86)\Atmel\Studio7.0\Extensions\Application\AvrGCC.dll".

Task "RunOutputFileVerifyTask"

Program Memory Usage : 178 bytes 0,5 % Full

Data Memory Usage : 0 bytes 0,0 % Full

Warning: Memory usage estimation may not be accurate if there are sections other than .text sections in ELF file

Done executing task "RunOutputFileVerifyTask".

Done building target "CoreBuild" in project "GCCApplication.cproj".

Target "PostBuildEvent" skipped, due to false condition: ("\$(PostBuildEvent)" != "") was evaluated as ("").

Target "Build" in file "C:\Program Files (x86)\Atmel\Studio7.0\Avr\comm.targets" from project "C:\Users\higuely\Documents\Atmel Studio7.0\Simulation\GCCApplication\GCCApplication.cproj" (entry point):

Done building target "Build" in project "GCCApplication.cproj".

Done building project "GCCApplication.cproj".

Build succeeded.

***** Build: 1 succeeded or up-to-date, 0 failed, 0 skipped *****

El código se ejecutó y compiló sin ningún error

E3:

Tabla 5. Correspondencia C - ensamblador (para completar)

Línea en C	Instrucción(es) AVR	Ciclos
DDRB = (1 << DDB5) (1 << DDB0);	in, ori, out	3
DDRD &= ~(1 << DDD2);	in, andi, out	3
PORTD = (1 << PORTD2);	in, ori, out	3
if (PIND & (1 << PIND2))	sbis, rjmp	2
PORTB &= ~(1 << PORTB5);	in, andi, out, rjmp	4
PORTB = (1 << PORTB5);	in, ori, out	3
PORTB ^= (1 << PORTB0);	in, ldi, eor, out	4

E4:

Simulation (Debugging) - Microchip Studio

File Edit View Visuals ASF Project Build Debug Tools Window Help

1545 31°

Simulation

main

int main(void)

Registers

Value

Type

Initial

Memory 4

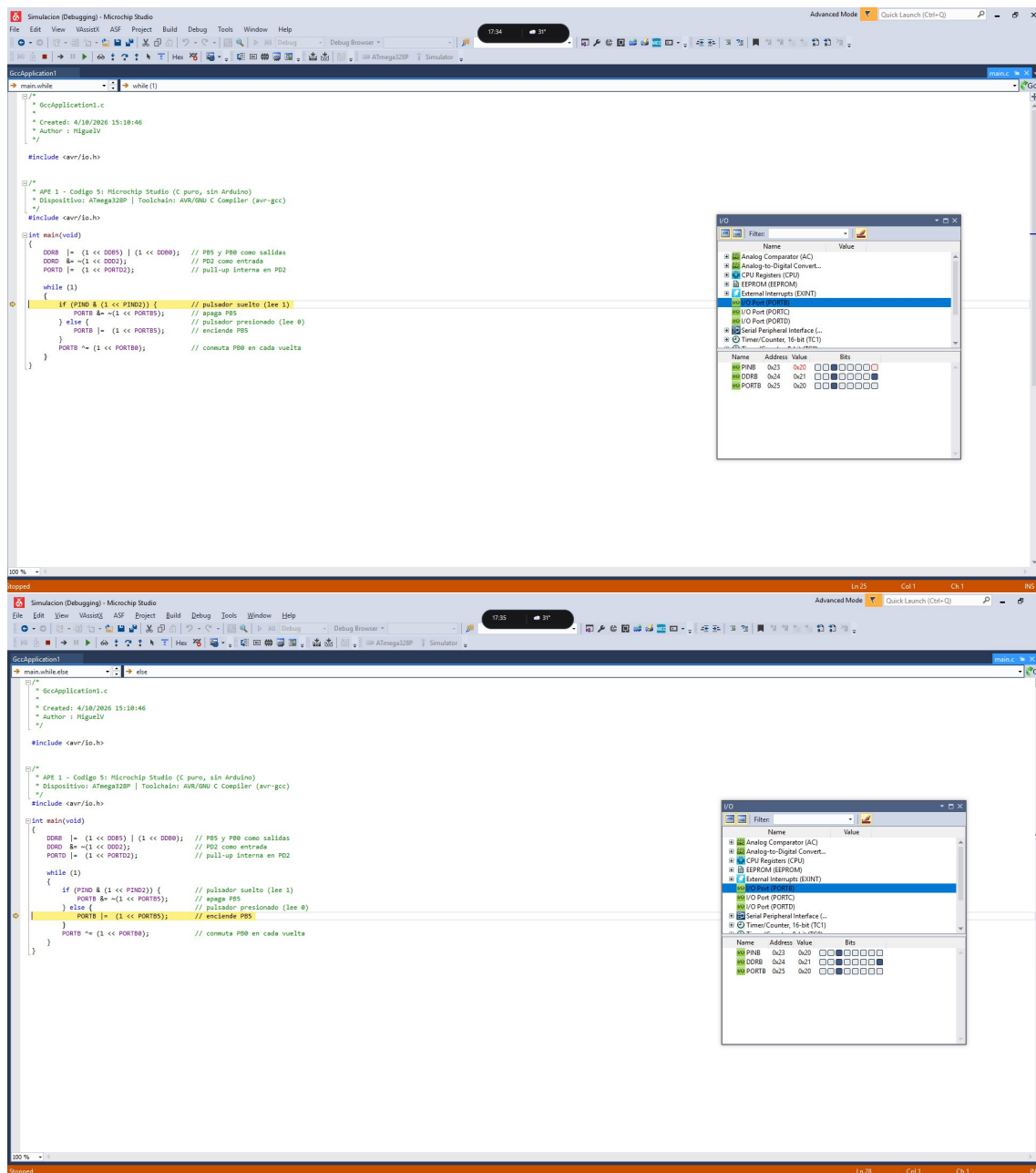
prog FLASH

Address: 0x00000000

Columns: Auto

Ready

E5:



The image displays two screenshots of the Microchip Studio IDE, showing the simulation of an AVR microcontroller program. The top screenshot shows the code in the 'main.while' loop, and the bottom screenshot shows the code in the 'main.while.else' loop. Both screenshots include a code editor on the left and an I/O window on the right.

Top Screenshot (main.while):

```

//
// * Application1.c
// Created: 4/18/2020 15:18:46
// Author : MiguelV
//
#include <avr/io.h>

/*
 * APE 1 - Código 5: Microchip Studio (C puro, sin Arduino)
 * Dispositivo: ATmega328P | Toolchain: AVR/GNU C Compiler (avr-gcc)
 */
#include <avr/io.h>

int main(void)
{
    DDRA |= (1 << DDRA5) | (1 << DDRA0); // PPS y PDB como salidas
    DDRA |= (1 << DDRA5); // PDB como entrada
    PORTD |= (1 << PORTD5); // pull-up interna en PDB

    while (1)
    {
        if (PIND & (1 << PIND5)) { // pulsador suelto (lee 1)
            PORTB |= (1 << PORTB5); // apaga PPS
        } else { // pulsador presionado (lee 0)
            PORTB |= (1 << PORTB5); // enciende PPS
        }
        PORTD ^= (1 << PORTD5); // conmuta PDB en cada vuelta
    }
}
    
```

I/O Window (Top):

Name	Address	Value	Bits
PINB	0x23	0x20	00000000
DDRB	0x24	0x21	00000000
PORTB	0x25	0x20	00000000

Bottom Screenshot (main.while.else):

```

//
// * Application1.c
// Created: 4/18/2020 15:18:46
// Author : MiguelV
//
#include <avr/io.h>

/*
 * APE 1 - Código 5: Microchip Studio (C puro, sin Arduino)
 * Dispositivo: ATmega328P | Toolchain: AVR/GNU C Compiler (avr-gcc)
 */
#include <avr/io.h>

int main(void)
{
    DDRA |= (1 << DDRA5) | (1 << DDRA0); // PPS y PDB como salidas
    DDRA |= (1 << DDRA5); // PDB como entrada
    PORTD |= (1 << PORTD5); // pull-up interna en PDB

    while (1)
    {
        if (PIND & (1 << PIND5)) { // pulsador suelto (lee 1)
            PORTB |= (1 << PORTB5); // apaga PPS
        } else { // pulsador presionado (lee 0)
            PORTB |= (1 << PORTB5); // enciende PPS
        }
        PORTD ^= (1 << PORTD5); // conmuta PDB en cada vuelta
    }
}
    
```

I/O Window (Bottom):

Name	Address	Value	Bits
PINB	0x23	0x20	00000000
DDRB	0x24	0x21	00000000
PORTB	0x25	0x20	00000000



1859

UNL

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

The image displays two screenshots of the Microchip Studio IDE, showing the development of a C program for an AVR microcontroller. The code is written in C and uses the AVR/GNU C Compiler (avr-gcc). The program is titled "main.while" and is located in the "GccApplication1" project.

Top Screenshot: The code is in the "main.while" file. It includes the "avr/io.h" header and defines the "main" function. The code sets up the I/O ports (DDRB, DDRC, PORTC) and enters a while loop. Inside the loop, it checks if the pushbutton is pressed (PINB & (1 << PINB2)) and toggles the LED (PORTB &= (1 << PORTB5)).

Bottom Screenshot: The code is the same as the top screenshot, but the comment for the pushbutton state is changed from "Pulsador suelto (lee 1)" to "Pulsador presionado (lee 0)".

I/O View: The I/O View window is open, showing the I/O Port (PORTC) selected. The table below shows the I/O port configuration:

Name	Address	Value	Bits
PINB	0x27	0x20	00000000
DDRB	0x24	0x21	00000000
PORTB	0x23	0x21	00000000

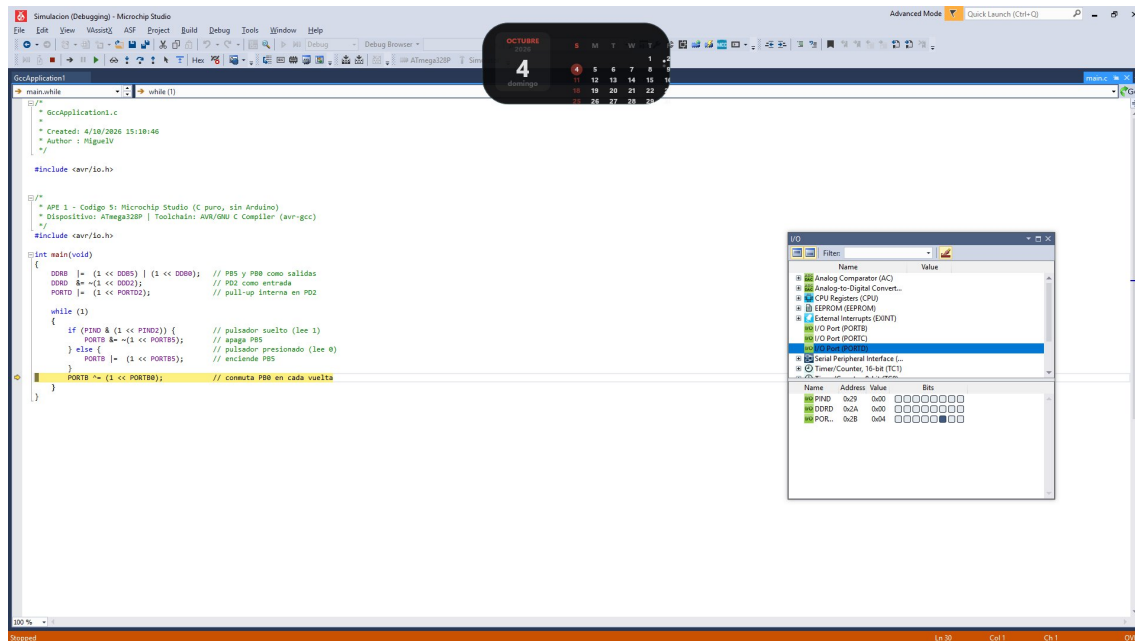


1859



Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación



E6:

7. Preguntas de Control

1. ¿Qué diferencia a un microcontrolador de un microprocesador? Enumere los bloques del ATmega328P que identificó en la práctica.

Un microprocesador como el de mi compu contiene solo la unidad de procesamiento (CPU) y necesita que le conectes memoria y periféricos externos para funcionar. Un microcontrolador como el ATmega328P es una "computadora en un solo chip": ya integra la CPU, la memoria (RAM y Flash) y los puertos de entrada/salida (I/O) en la misma pastilla.

2. ¿Qué implica que el ATmega328P tenga arquitectura Harvard? Relacione su respuesta con el uso de la macro F("...") en el Código

Implica que el microcontrolador tiene memorias y buses de comunicación físicamente separados para los Datos (RAM) y para las Instrucciones/Programa (Flash). Esto le permite a la CPU leer una instrucción y un dato al mismo tiempo, haciéndolo más rápido y por lo que la memoria RAM del ATmega328P es pequeñísima (apenas 2 KB). Por defecto, al imprimir texto normal, el sistema copia ese texto de la Flash a la RAM, desperdiciando espacio. La macro F() aprovecha la separación de la arquitectura Harvard para decirle al sistema: "deja este texto guardado en la memoria Flash y léelo desde ahí", ahorrando RAM valiosa.

3. Explique por qué `PORTB = (1 << PORTB5);` puede alterar otras salidas y `PORTB |= (1 << PORTB5);` no, usando lo observado en los pasos B4 y C6.

La asignación directa (`PORTB = valor`) altera todos los bits del registro PORTB al mismo tiempo, modificando pines que no querías cambiar. Por otro lado, usar el operador `|=` realiza una operación de lectura-modificación-escritura que permite cambiar solo el bit deseado (como PB5) y mantener el estado actual de los demás bits intacto.

4. Para usar PD2 como entrada con pull-up, ¿qué valores deben tener `DDRD2` y `PORTD2`? ¿Qué se lee en `PIND2` con el pulsador suelto y



presionado, y por qué?

Según tu Tabla 4 y la sección C7, el bit DDRx debe ser 0 y el bit PORTx debe ser 1. Cuando el pulsador está suelto, el bit PINx leído es 1.

Cuando el pulsador está presionado, el bit PINx leído es 0.

5. Calcule a mano $0b11001100 \& 0b10101010$, | y ^, y verifique sus resultados con el Monitor Serie.

Me dieron los mismos resultados al calcular a mano y en el monitor serie.

6. Si `a` es de tipo `uint8_t`, ¿por qué `Serial.println(~a, BIN)` imprime muchos más de 8 bits? ¿Cómo lo corrige el Código 3?

Como se observa en la consola de tu simulación (Modo 1), imprimir ~a sin "cast" provoca que se muestren más bits (se imprime 111111111111111111111100110011). El Código 3 lo corrige aplicando un cast de esta forma: (uint8_t)~a.

7. Con los datos de la Parte D, explique por qué `digitalWrite()` es más lento que el acceso directo a registros. ¿Cuándo conviene cada enfoque (portabilidad frente a rendimiento)?

Según las capturas de la Parte D1, `digitalWrite()` tomó en promedio 3.395 us por escritura, mientras que el acceso directo a los registros tomó solo 0.220 us por escritura. El acceso a registros es muchísimo más rápido porque escribe directamente en el hardware.

8. ¿Qué instrucción AVR generó $PORTB \leftarrow (1 \ll PORTB5)$; y cuántos ciclos toma?

De acuerdo con la "Tabla 5" en el paso E3, generé las instrucciones in, ori, out y toma 3 ciclos. (Nota: En la Tabla 5 de la parte D2, también indicaste que se generó mediante sbi 0x5,5 tomando 2 ciclos).

¿Por qué `PORTB ^= (1 << PORTB0)`; no se tradujo a una sola instrucción?

Según la Tabla 5 del paso E3, esta línea en C se tradujo a las instrucciones AVR `in`, `ldi`, `eor`, `out`, tomando un total de 4 ciclos. No se hizo en una sola instrucción porque requiere varios pasos a nivel ensamblador para aplicar la operación lógica XOR (cargar el valor, realizar la operación exclusiva y sacarlo).

9. **Sostenibilidad:** indique dos formas en que lo trabajado (simular antes de comprar o montar, acceso eficiente a los periféricos) contribuye a reducir el consumo energético o los residuos electrónicos.

Simular en Wokwi antes de armar el circuito real evita que quemes componentes físicos (LEDs, resistencias o el propio microcontrolador) por cortocircuitos o malas conexiones, lo que significa menos placas arruinadas en la basura.

- 10. Ventajas y limitaciones de simular en Wokwi frente al hardware real (rebote del pulsador, límites de corriente, tiempos).**

Es totalmente seguro e ideal para probar la lógica. No hay cables falsos que hagan mal contacto, no necesitas comprar hardware para aprender y puedes compartir el circuito con un enlace.



8. Conclusiones

- Durante esta práctica comprobé que manipular directamente los registros (DDR, PORT, PIN) del ATmega328P es drásticamente más rápido que usar la API nativa de Arduino. Medir los tiempos de ejecución me demostró que el acceso a registros toma apenas 0.220 microsegundos por escritura, frente a los 3.395 microsegundos de digitalWrite, un rendimiento que será clave cuando necesite programar aplicaciones donde el tiempo de respuesta sea crítico.
- El uso del simulador y de Microchip Studio me permitió entender lo que realmente ocurre dentro del microcontrolador a nivel de hardware. Observar cómo mis líneas de código en C se traducen a instrucciones específicas de ensamblador (como sbi, cbi, in, out), y ver cómo cambian los bits en la memoria paso a paso, me dio una perspectiva mucho más profunda sobre cómo optimizar mis programas en lugar de solo escribir código a ciegas.

9. Bibliografía

[1] W. P. Chamba Zaragocín, E. R. Guamán Quinche, F. J. Alvarez Pineda, J. O. Guamán Quinche, G. B. Zaragocín Arrobo y G. Á. Valarezo Benítez, Análisis y diseño preliminar de software basado en el modelado de requisitos y casos de uso. Un enfoque teórico y práctico. Loja, Ecuador: Universidad Nacional de Loja, 2025. Disponible en: <https://unl.edu.ec/investigacion/produccion-cientifica/analisis-y-diseno-preliminario-de-software-basado-en-el-modelado>